

32555 – Fundamentals of Software Development



Welcome to 32555 – Fundamentals of Software Development

Your Teaching Team:

Dr. Yining Hu | Subject Coordinator

Yining is a Lecturer in Software Engineering. She works with many SMEs to develop software and AI solutions through student and research projects at UTS.

Contact: yining.hu@uts.edu.au

Dr. Xiaojie (Zoe) Lin | Tutor

Zoe is a full-stack software engineer with an interest in data analysis, specializing in JAVA, R, Python, JavaScript, PHP, Bash, and MATLAB; familiar with AWS.

Contact: Xiaojie.Lin@uts.edu.au

Dr. Farshid Keivanian | Tutor

Dr. Farshid Keivanian is an experienced university lecturer and researcher with a PhD in Information Technology. He has over 15 years of teaching experience across Australian institutions, specialising in software engineering, data science, AI, and cybersecurity.

Contact: Farshid.Keivanian@uts.edu.au

Yong Pov | Tutor

Yong is a full-stack software engineer with experience in React, JavaScript, TypeScript, Java, Python, SQL, and AI-assisted learning platforms.

Contact: Yong.Pov@uts.edu.au

Welcome to 32555 – Fundamentals of Software Development

Cmp1 #	Location	Time	Tutor
01	CB11.B1.400	Fri 16:00-18:00	Xiaojie (Zoe) Lin
07	CB11.B1.401	Wed 10:00-12:00	Farshid Keivanian
08	CB11.B1.400	Fri 12:00-14:00	Yong Pov
09	CB11.B1.103	Wed 16:00-18:00	Farshid Keivanian
10	CB11.B1.401	Wed 12:00-14:00	Farshid Keivanian

Classes and Objects

What is a Class?

- A class is a blueprint for creating objects.
- It defines attributes (data) and methods (behaviours).
- Class names are nouns (e.g., Student, Account, CreditCard).

Classes and Objects

What is an Object?

- An object is a real-world instance of a class.
- Objects exist at runtime and store their own data.

Classes and Objects

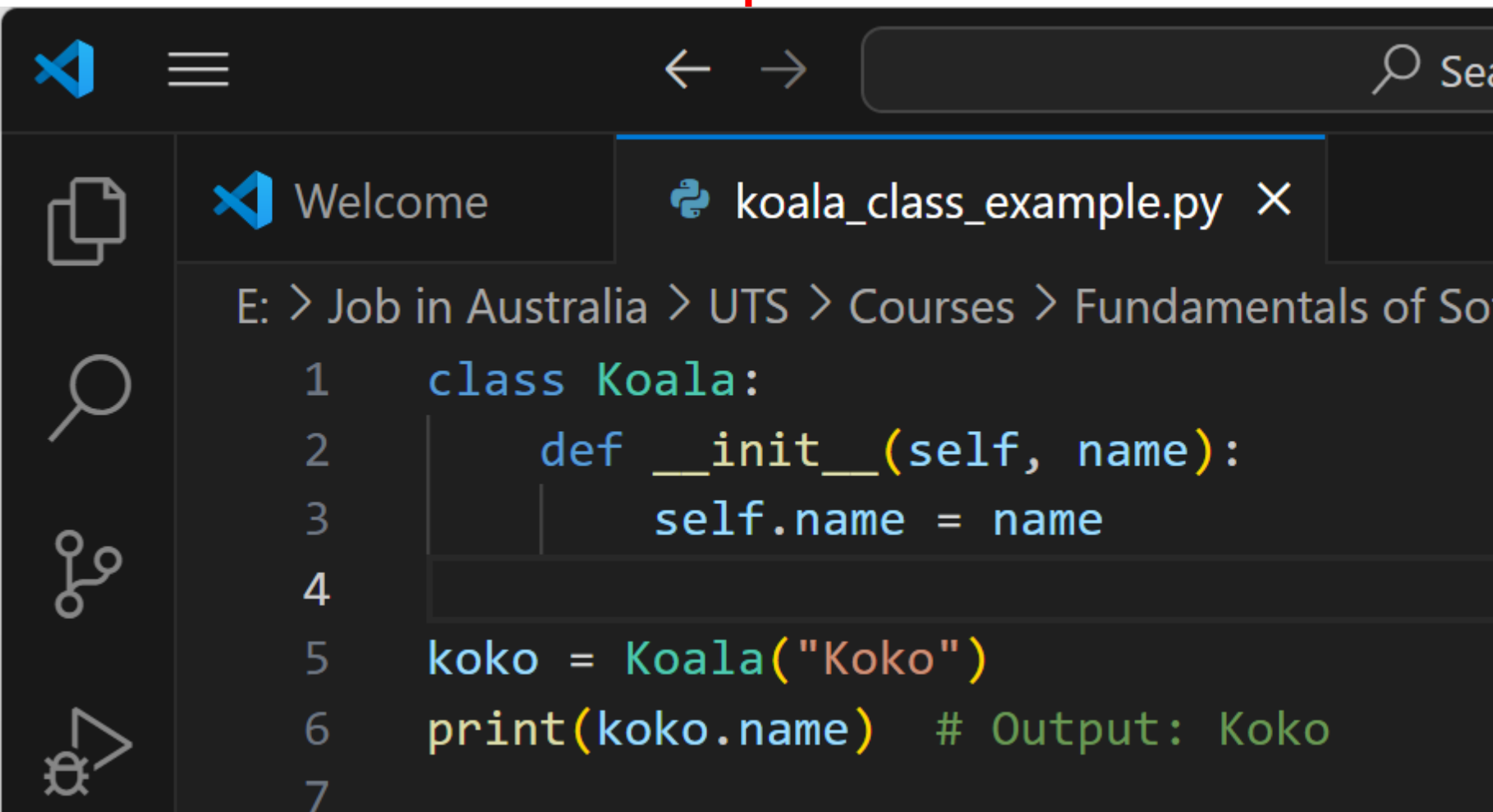
Real-World Analogy:

- **Class** = Recipe
- **Object** = Cake made using that recipe

Australian Example:

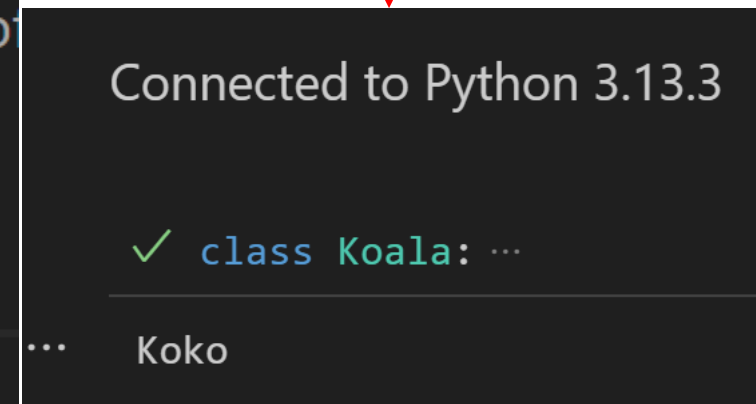
- **Class:** Koala
- **Object:** A koala named “Koko” at Taronga Zoo

Koala Class Example in Python



The screenshot shows a code editor with a dark theme. The file name is 'koala_class_example.py'. The code is as follows:

```
1 class Koala:
2     def __init__(self, name):
3         self.name = name
4
5 koko = Koala("Koko")
6 print(koko.name) # Output: Koko
7
```



The terminal window shows the output of the Python code. It is connected to Python 3.13.3. The output is:

```
✓ class Koala: ...
... Koko
```

This code defines a class called Koala with a name attribute. It then creates an object koko from the class and prints its name.

Activity 1: Add a New Attribute and Print It

Task:

Modify the Koala class to include an additional attribute called age. When creating a koala object, assign both name and age, and then print both.

```
class Koala:
    def __init__(self, name, age):
        self.name = name
        self.age = age

koko = Koala("Koko", 4)
print(f>Name: {koko.name}, Age: {koko.age}")
```

Learning Outcome: Understanding how to add and initialize multiple attributes.

Activity 2: Add a Method to Display a Greeting

Task:

Add a method to the Koala class called greet() that prints a custom message using the koala's name. Then call this method on an object.

```
class Koala:
    def __init__(self, name):
        self.name = name

    def greet(self):
        print(f"G'day! I'm {self.name}, the koala.")
```

```
koko = Koala("Koko")
koko.greet()
```

Learning Outcome: Understanding how to define and use instance methods.

Object-Oriented Paradigm Principles

What is OOP?

- It's a way of writing code by thinking in terms of real-world objects.
- These objects can have data (like properties) and actions (like behaviours).
- OOP helps us build programs that are easier to understand and maintain.

Object-Oriented Paradigm Principles

4 Main Ideas in OOP (AEIP)

Principle	What it Means (Simple)	Real-Life Example
Abstraction	Hide the messy stuff and show only the basics	Coffee machine – you press a button, not worry how it works
Encapsulation	Lock the data inside and control who sees or changes it	Medicine sealed in a capsule
Inheritance	Share code from one class to another	Kids inherit their parents' eye colour
Polymorphism	One word, different actions depending on the object	“Play” button on Spotify vs. radio

Abstraction

What is Abstraction?

- It means hiding the complicated parts of something.
- You only show what's needed, not how it works inside.
- It helps people use things easily without getting confused by the details

Abstraction

Two Types of Abstraction:

- **Data abstraction** → Hide the variable itself, just show the value
(e.g., you see your bank balance, not how it's stored in code)
- **Function abstraction** → Hide the code, just use the function
(e.g., use `withdraw()` without seeing how it works inside)

Abstraction

Real-World Example (Australia):

When you use an ATM, you don't care how it connects to your bank.

You just enter your PIN, choose an amount, and get your cash.

That's abstraction — using it without knowing the details.

Using Abstraction in Java with Animal Example

```
abstract class Animal {  
    abstract void makeSound();  
}  
class Dog extends Animal {  
    void makeSound() {  
        System.out.println("Bark!");  
    }  
    public static void main(String[] args) {  
        Dog dog = new Dog();        // Create an object of Dog  
        dog.makeSound();             // Call the method  
    }  
}
```

This code shows how abstraction works in Java using an abstract class called Animal.

The Dog class extends Animal and provides its own version of the makeSound() method.

The main() method creates a Dog object and calls its sound method to print "Bark!".



```
▼ TERMINAL  
● PS C:\Users\gaming 15> java -d jdk-21\bin\java.exe -cp .\src\test\bin 'Dog'  
Bark!
```

Using Abstraction in Java with Animal Example

```
abstract class Animal {  
    abstract void makeSound();  
}  
class Dog extends Animal {  
    void makeSound() {  
        System.out.println("Bark!");  
    }  
    public static void main(String[] args) {  
        Dog dog = new Dog();           // Create an object of Dog  
        dog.makeSound();                // Call the method  
    }  
}
```

In short:

It hides the general idea (makeSound) in the Animal class and lets the Dog class decide how to make the sound — this is abstraction in action.



```
▼ TERMINAL  
● PS C:\Users\gaming 15> java -Djava.class.path=. -jar 'C:\Program Files\Java\jdk-21\bin\java.exe' '-cp' 'C:\Users\gaming\AppData\Local\Temp\vscodesws_60f\src\bin' 'Dog'  
Bark!
```


Encapsulation

What is Encapsulation?

- Put data and functions together in one class
- Hide the details (like private variables)
- Let other parts of the program access the data safely using public methods
- Keeps things safe, simple, and easy to manage

Encapsulation

Real-Life Analogy:

A capsule pill hides the medicine inside. You can take it, but not open or change it.

Australian Example:

You can see your Opal card balance — but you can't change the balance yourself.

Encapsulation Example Account in Python

```
class Account:
    def __init__(self, balance):
        self.__balance = balance    # Hidden (private)

    def get_balance(self):
        return self.__balance      # Safe way to access balance
```

This code shows **encapsulation** by keeping the balance private and allowing access through a safe `get_balance()` method.

Inheritance

What is Inheritance?

- One class gets features (methods and variables) from another
- Helps you reuse code instead of writing it again
- Makes programs easier to manage and extend

Inheritance

Real-Life Analogy:

A Cashier is a Person. A Manager is also a Person.

They all share things like name and age but have their own tasks too.

Rules:

- A child class can use everything from its parent class
- It can also change or add its own methods

Inheritance kangaroo example in Java

```
class Animal {  
    void eat() {  
        System.out.println("Eating...");  
    }  
}  
  
class Kangaroo extends Animal {  
    void jump() {  
        System.out.println("Jumping...");  
    }  
}
```

This Java code shows **inheritance**, where Kangaroo inherits the eat() method from the Animal class and adds its own jump() method.

Inheritance kangaroo example in Java

```
class Animal {  
    void eat() {  
        System.out.println("Eating...");  
    }  
}  
  
class Kangaroo extends Animal {  
    void jump() {  
        System.out.println("Jumping...");  
    }  
}
```

This code defines the classes, but does not run anything because there's no main() method.

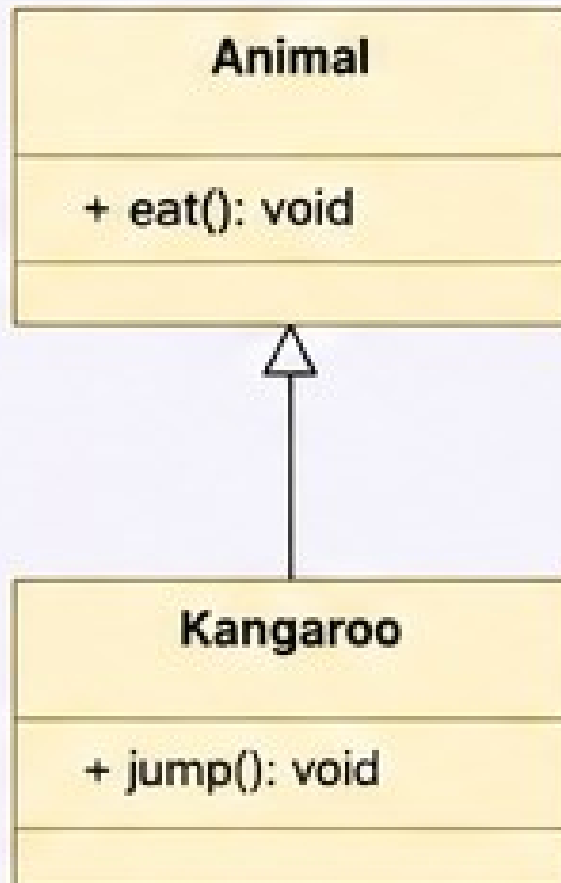
Inheritance kangaroo example in Java

```
public class Main {  
    public static void main(String[] args) {  
        Kangaroo k = new Kangaroo();  
        k.eat();  
        k.jump();  
    }  
}
```

To Produce Output,
Add a main() Method:

UML Class Diagram for the Java inheritance example with Animal and Kangaroo

Completed Diagram



Explanation

- Animal is the superclass (parent).
- Kangaroo is the subclass (child).
- Kangaroo inherits `eat()` from Animal.
- Kangaroo also has its own method `jump()`.
- The hollow triangle points to the superclass (Animal).

UML Notation



UML Class



Generalization (Inheritance)

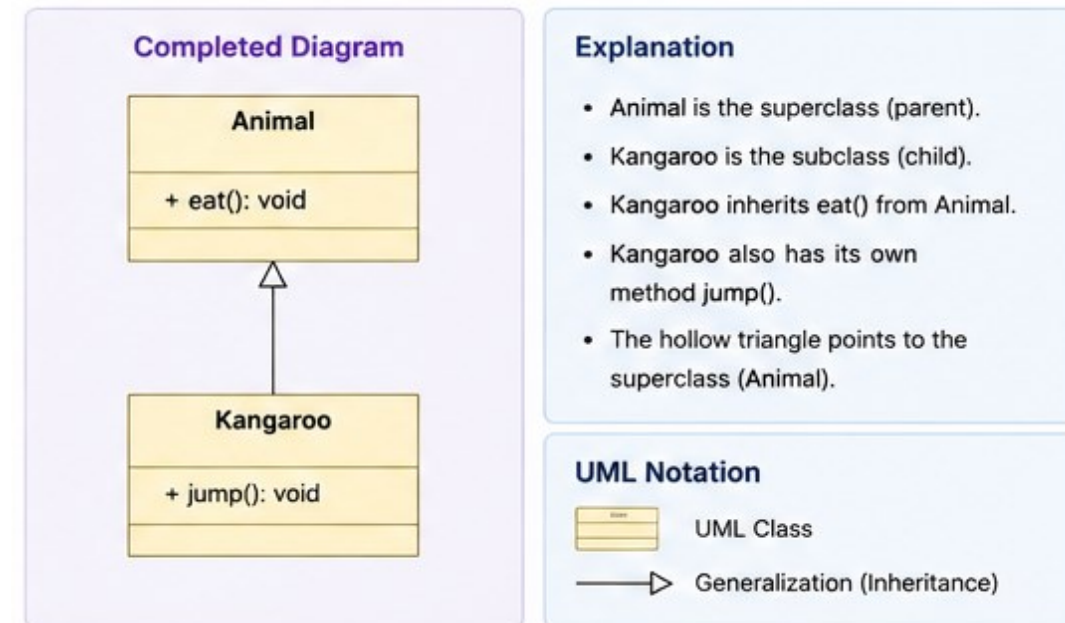
UML Class Diagram for the Java inheritance example with Animal and Kangaroo

In UML class diagrams, it is better to include the method name, its parameters, and its return type using this format:

+ methodName(parameterList): returnType

So we write

+ jump(): void



UML Class Diagram for the Java inheritance example with Animal and Kangaroo

It means:

- + → The method is public (accessible from outside the class)
- jump() → This is the method name with no parameters
- void → The method does not return any value

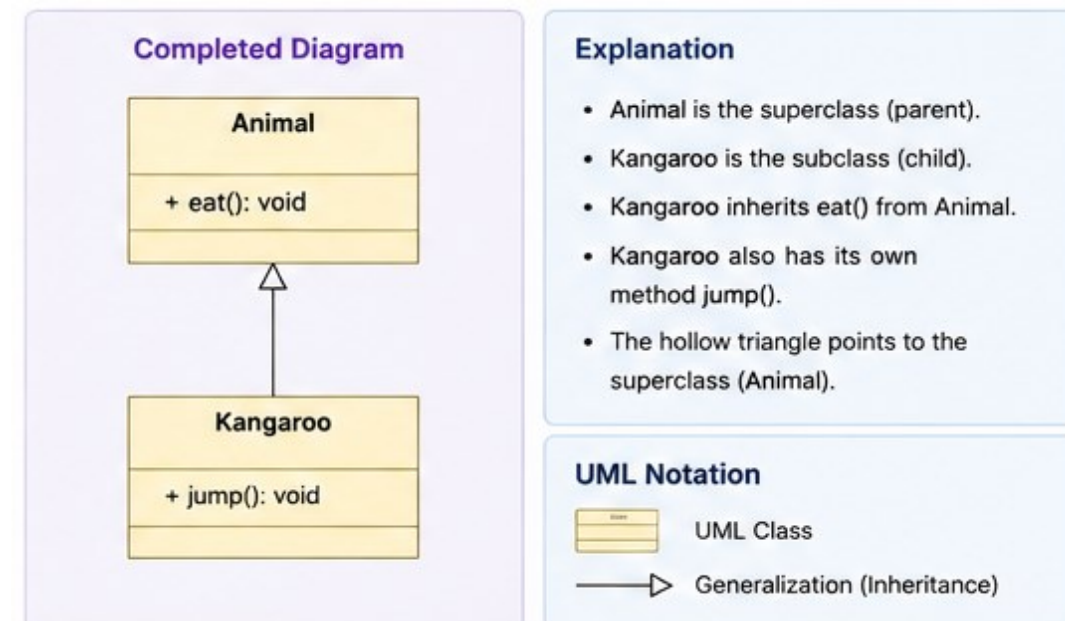
Same for eat():

- It just prints "Eating...", it doesn't return anything.
- So we label it void (same as in Java code).

+ methodName(parameterList): returnType

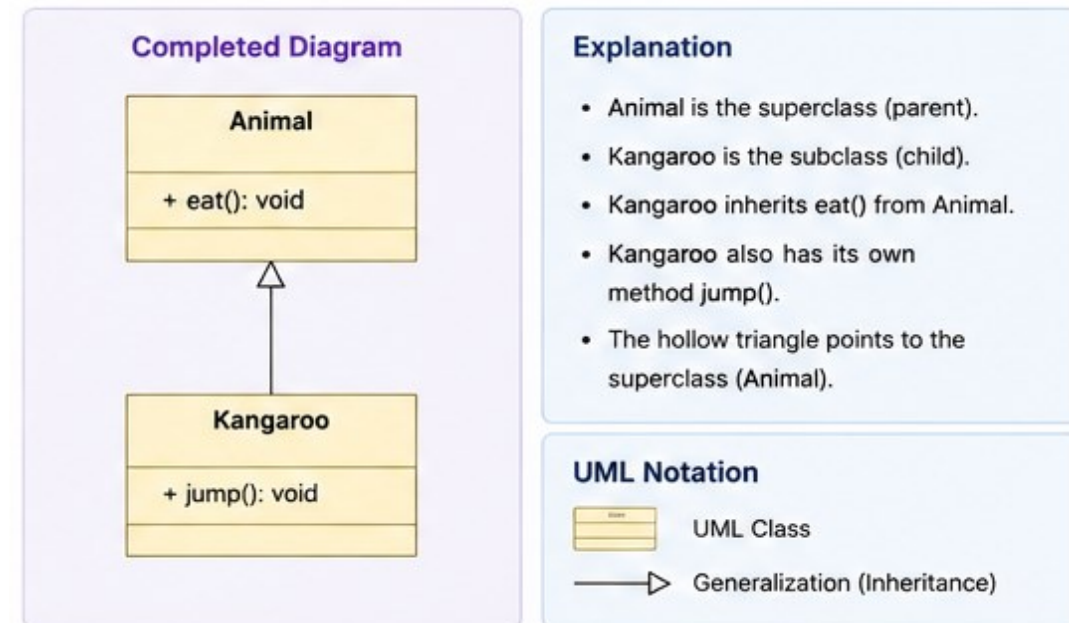
So we write

+ jump(): void



UML Class Diagram for the Java inheritance example with Animal and Kangaroo

Summary: We use void in UML for the same reason we use it in Java — to show that the method does something but does not give anything back (like a number or string).



Key Differences in UML When Representing Python

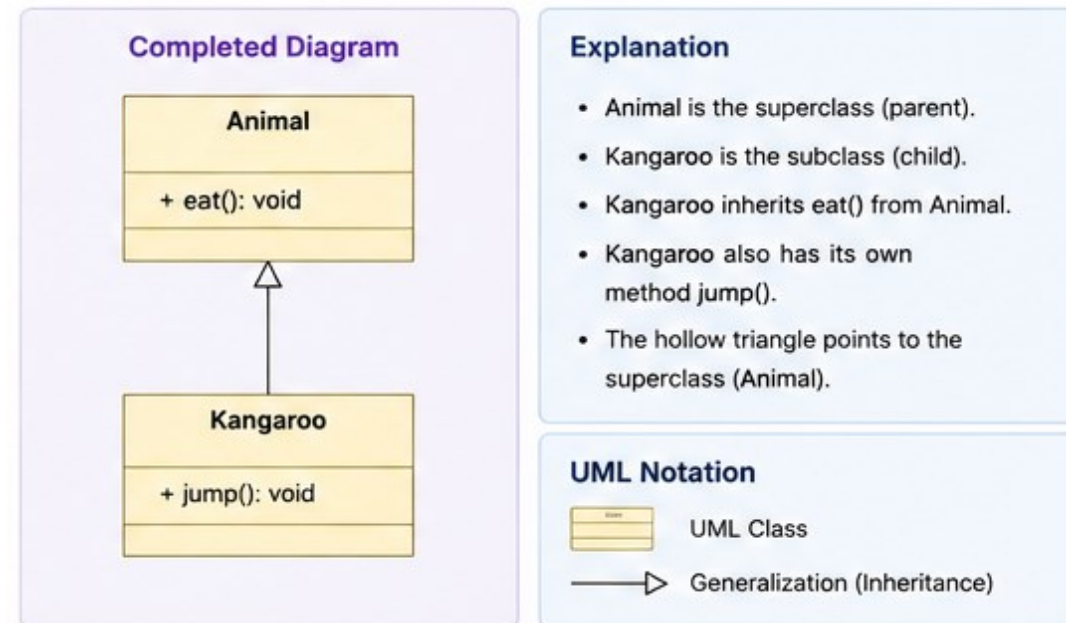
The structure of a UML class diagram is the same, whether we're using Java, Python, or another object-oriented language.

Key differences in UML when representing Python:

Feature	Java Style	Python Style
Access modifiers	+ (public), - (private), # (protected)	Python has no true enforcement — use <code>_</code> (protected), <code>__</code> (private by convention)
Return types	Required (e.g., <code>: void</code> , <code>: int</code>)	Optional — Python is dynamically typed
Constructor name	<code>ClassName()</code>	<code>__init__()</code>
Static vs instance	Explicit in Java (<code>static</code>)	Not shown in UML for Python unless needed

UML Class Diagram (Python-style, simplified)

- UML diagrams work across all languages
- You can keep using the same diagrams for Python, just ignore strict return types or access modifiers
- Use `__init__()` to represent constructors in Python



Polymorphism

What is Polymorphism?

- One function, many behaviours
- The same method name does different things depending on which object calls it
- Helps write flexible and reusable code

Polymorphism

Real-Life Analogy:

You say “Open” — a fridge, a file, and a door will all open, but in different ways

Australian Example:

The method `calculateInterest()` gives different results for:

- A MyGov savings account
- A Centrelink credit benefit

Polymorphism Bird Example in Python

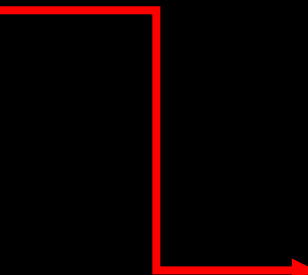
```
class Bird:
    def make_sound(self):
        print("Tweet!")

class Emu(Bird):
    def make_sound(self):
        print("Grunt!")

def speak(bird):
    bird.make_sound()

speak(Bird())      # Output: Tweet!
speak(Emu())       # Output: Grunt!
```

This Python code shows polymorphism, where different bird objects respond to the same `make_sound()` method in their own unique way.



Interrupt | X Clear All | ↺ Restart | ... Python 3.13.3

```
✓ class Bird: ...
```

```
... Tweet!
    Grunt!
```

Object-Oriented Design (OOD)

What is OOD?

- It's how we plan software using objects, their data, and their actions
- Helps make code easier to build, test, and maintain

Object-Oriented Design (OOD)

Key Steps:

1. Find the classes (from real use cases)
2. List their attributes (nouns) and methods (verbs)
3. Show how they relate
4. Draw a UML class diagram

Object-Oriented Design (OOD)

Australian Example:

Designing a MyGov login system:

- Classes: User, Account, Authenticator
- Methods: login(), validateOTP()

"A Customer uses the deposit() method, but it's defined in the Account class."

Object-Oriented Design (OOD)

Australian Example:

Designing a MyGov login system:

- Classes: User, Account, Authenticator
- Methods: login(), validateOTP()

"A Customer uses the deposit() method, but it's defined in the Account class."

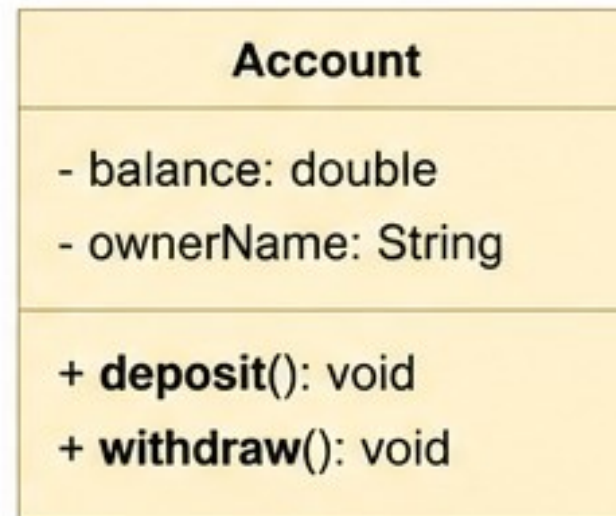
Understanding UML Class Diagrams (Account Example) – Java Style Notation

What is UML?

- UML = Unified Modeling Language
- Used to draw diagrams that show how classes are built and how they relate

- means private

+ means public



Explanation

- Top compartment: Class name (Account)
- Middle compartment: Attributes (data)
- Bottom compartment: Operations (methods)
- “-” means private
- “+” means public

Recommended Tools for Drawing UML Diagrams

Tools You Can Use:

- [Draw.io](https://draw.io)
- Visual Paradigm (free)
- Lucidchart
- Microsoft Visio

These tools help you create class diagrams, use case diagrams, and more—free and beginner-friendly.

Tool	Description	Platform	Best For	Link
Draw.io (diagrams.net)	Intuitive, browser-based diagramming tool with UML templates.	Web, Desktop	Class Diagrams, Use Case Diagrams, Flowcharts	https://draw.io
Lucidchart (Free Plan)	Easy-to-use interface with real-time collaboration and UML templates.	Web	UML diagrams, Use Case design, Group work	https://www.lucidchart.com
Visual Paradigm (Community Edition)	Professional UML and system modeling tool with rich diagram support.	Desktop (Windows/macOS/Linux)	Class Diagrams, Sequence Diagrams, DFDs	https://www.visual-paradigm.com/download/community.jsp
StarUML (Free Trial or License)	Designed for software engineering students and professionals.	Desktop	Full UML diagram suite (recommended in course)	https://staruml.io
Creately (Free Tier)	Collaborative visual workspace with multiple UML templates.	Web	Use Cases, Class Diagrams, ERDs	https://creately.com

Recommendation for Students

- Beginner-friendly: Start with Draw.io or Lucidchart.
- Professional-level UML modelling: Use StarUML or Visual Paradigm.
- Offline work: Prefer Visual Paradigm or StarUML desktop versions.
- Collaboration features: Use Lucidchart or Creately (great for group projects).

Using Microsoft Visio for UML Diagrams

Microsoft Visio is a powerful diagramming tool that can be used to draw a variety of system and software models, including UML diagrams.

Visio is suitable for creating:

- Use Case Diagrams
- Class Diagrams
- Activity Diagrams
- Sequence Diagrams

Using Microsoft Visio for UML Diagrams

However, please note:

Feature	Details
Installation	Part of Microsoft 365 (Pro Plus or Enterprise editions)
License	May require separate license or institutional access
Suitability for Beginners	Moderate (not as intuitive as Draw.io or StarUML)
Export Options	Easily export to PDF or image formats
UML Templates	Available via built-in or downloadable stencils

Using Microsoft Visio for UML Diagrams

If your university provides Microsoft 365 for Education:

1. Visit your university's Office 365 portal, e.g.
<https://www.office.com>
2. Sign in using your University email.
3. Click on "Install Office".
4. If Visio is included in your license, you will see an option to install Visio under "Other installs".

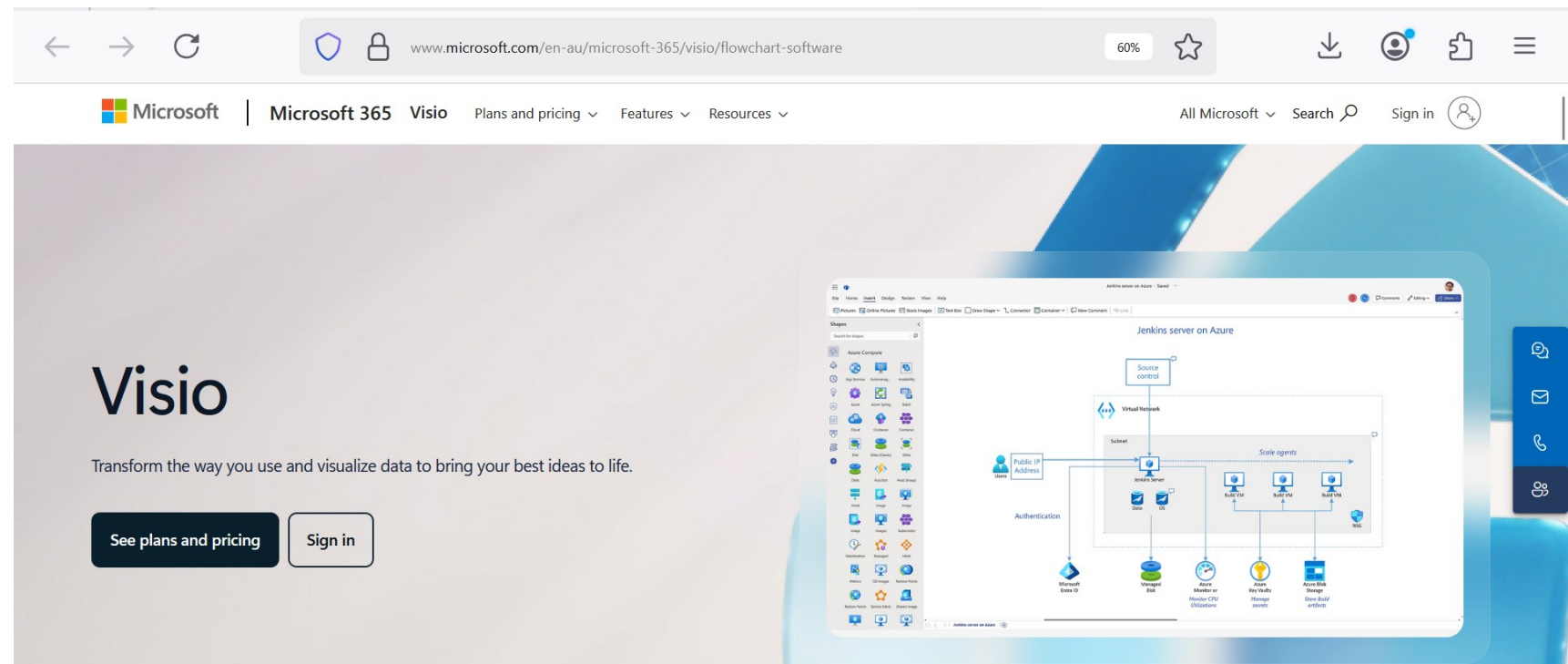
Not all student accounts include Visio by default. Check with your university IT or library services.

Using Microsoft Visio for UML Diagrams

To download Visio as a standalone app (if not included in your student license):

<https://www.microsoft.com/en-au/microsoft-365/visio/flowchart-software>

This is the official Microsoft Visio page. Scroll down to compare Visio Plan 1 (web only) and Plan 2 (includes desktop app).



Using Microsoft Visio for UML Diagrams

Tip for Students

- Visio Plan 1 is browser-based and lightweight (monthly subscription).
- Visio Plan 2 includes the full desktop version and is ideal for UML diagrams and complex modelling tasks.

Lab 2 Exercises

Complete the Lab 2 exercises as a team

(form groups of 3-4 students)

Lab 2 continues from Lab 1

Lab 2 Exercises

Canvas >> Modules >> Lab 2: Object-Oriented Design

Lab 2: Object-Oriented Design

Lab 2 aims to further enhance your software design knowledge through activities that cover the following key topics:

- Object-oriented design
 - Organizing a program into classes
 - Identify the design layers (abstraction, implementation, interactions)
 - Identify inheritance within the system
- Class diagram
 - Visual the OO Design into UML class diagram representation

Lab 2 Exercises

Lab Case Study: The DeskBankApp system

Your team has now completed the requirements analysis for the new Desk Banking System (lab 1) which includes requirements analysis and testing and use case modelling. The next step is to conduct object-oriented design and develop a UML class diagram for the DeskBankApp.

Lab 2 Exercises

Lab Activities: OOD – DeskBankApp system

Use Lab 1 requirements analysis and user case modelling to:

- Identify the classes and provide a list of the classes and explain in detail your classes selection.
- Identify the attributes of each class (attribute names, types, visibilities).
- Identify the operations (visibilities and potentially with input and output types).

Create a UML class diagram and concisely denote the follow:

- Class relationships and multiplicity.
- Attributes and operations and their visibility of each class.

Lab 2 Exercises

Transforming a use case diagram into a class diagram

“What data and operations are needed to support a certain user interaction?”:

1. Identify key nouns (things, roles, data entities) -> potential classes
2. Identify actors -> potential classes
3. Identify behaviors -> methods
4. Group related use cases -> distribute methods to the identified classes
5. Identify class relationships -> think about the interactions between actors and use cases

Lab 2 Exercises

Lab Tools:

The following tools are recommended to complete lab 2 activities:

- Download and use [Visual Paradigm Community Edition](#)
- Use the online drawing tool: [draw.io](#)

Feel free to use other tools of choice!

Lab 2 Exercises

Lab Case Study: **The DeskBankApp system**

Your team has now completed the requirements analysis for the new Desk Banking System (lab 1) which includes requirements analysis and testing and use case modelling. The next step is to conduct object-oriented design and develop a UML class diagram for the DeskBankApp.

Lab 2 Exercises

Lab Activities: OOD – DeskBankApp system

Use Lab 1 requirements analysis and user case modelling to:

- Identify the classes and provide a list of the classes and explain in detail your classes selection.
- Identify the attributes of each class (attribute names, types, visibilities).
- Identify the operations (visibilities and potentially with input and output types).

Create a UML class diagram and concisely denote the follow:

- Class relationships and multiplicity.
- Attributes and operations and their visibility of each class.

Lab 2 Practical Steps

Transforming a use case diagram into a class diagram

“What data and operations are needed to support a certain user interaction?”:

1. Identify key nouns (things, roles, data entities) -> potential classes
2. Identify actors -> potential classes
3. Identify behaviors -> methods
4. Group related use cases -> distribute methods to the identified classes
5. Identify class relationships -> think about the interactions between actors and use cases

Lab 2: Interactive Questions

Question 1) What is the goal of Lab 2?

Think about what we are trying to achieve after completing the requirements analysis.

The goal of Lab 2 is to convert the requirements and use case diagram into an Object-Oriented Design (OOD).

By the end of this lab, you should be able to:

- Identify classes
- Identify attributes
- Identify operations (methods)
- Identify relationships
- Draw a UML Class Diagram

Lab 2: Interactive Questions

Question 2) What is Object-Oriented Design (OOD)?

Object-Oriented Design (OOD) is the process of designing software using objects and classes.

Instead of thinking about individual functions, we think about:

- Objects
- Their data (attributes)
- Their behaviour (methods)
- How they work together

Example:

Instead of writing one large banking program,

we divide it into classes such as:

- Customer
- Account
- Transaction
- Manager

Lab 2: Interactive Questions

Question 3) Why do we organise a program into classes?

Organising a program into classes makes software:

- Easier to understand
- Easier to maintain
- Easier to reuse
- Easier to test
- Easier to extend

Example

Instead of storing everything in one program,
we separate responsibilities.

Customer manages customer information.

Account manages account information.

Transaction manages banking transactions.

Lab 2: Interactive Questions

Question 4) What are the design layers?

Software can be viewed in three design layers.

1. Abstraction

"What should the system do?"

Example

Customer

Account

Transaction

Lab 2: Interactive Questions

2. Implementation

"How is it programmed?"

Example

Java

Python

Methods

Variables

Lab 2: Interactive Questions

3. Interaction

"How do the objects communicate?"

Example

Customer → Account → Transaction

Lab 2: Interactive Questions

Question 5) What is a class?

A class is a blueprint for creating objects.

A class contains:

- Attributes (data)
- Operations (methods)

Example: Customer

Attributes

- customerID
- name
- email

Methods

- login()
- logout()
- viewAccounts()

Lab 2: Interactive Questions

Question 7) How do we identify classes?

Read the requirements carefully.

Look for important nouns.

Example

Requirement: Customer can deposit money into an account.

Possible classes

- Customer
- Account
- Transaction

Nouns often become classes.

Lab 2: Interactive Questions

Question 8) Which classes can we identify for DeskBankApp?

Possible classes include:

- Customer
- Account
- Transaction
- Manager
- Employee
- Branch

Depending on the design, additional supporting classes may also be identified.

Lab 2: Interactive Questions

Question 9) Why is "Deposit" not a class?

A deposit is an action, not a thing.

Classes usually represent things.

Methods usually represent actions.

Java example

```
public void deposit(double amount) {  
    balance = balance + amount;  
}
```

Python example

```
def deposit(self, amount):  
    self.balance = self.balance + amount
```

Here, deposit() is a method inside the Account class.

Remember:

- Account → class (thing)
- deposit() → method (action)
- amount → method parameter (input)

Lab 2: Interactive Questions

Question 10) How do we identify attributes?

Think about the information each class needs.

Example

Customer

- `customerID`
- `fullName`
- `email`
- `phone`

Account

- `accountNumber`
- `balance`
- `accountType`

Question 11) What is an attribute?

An attribute is data stored inside a class.

Attributes describe the objects created from the class.

Java example

```
class Account {  
    private String name;  
    private double balance;  
}
```

Python example

```
class Account:  
    def __init__(self, name, balance):  
        self.name = name  
        self.balance = balance
```

In both examples:

- name stores the account holder's name.
- balance stores the amount of money in the account.

Each Account object can have a different name and balance.

Lab 2: Interactive Questions

Question 12) What is a method (operation)?

A method describes what an object can do.

Example

Customer

`login()`

`logout()`

`viewAccounts()`

Account

`deposit()`

`withdraw()`

`getBalance()`

Methods are usually written using verbs.

Lab 2: Interactive Questions

Question 13) What is visibility?

Visibility controls who can access data or methods.

Symbol	Meaning
+	Public
-	Private
#	Protected

Example

```
- balance : double
```

```
+ deposit()
```

```
+ withdraw()
```

Private data protects the object.

Lab 2: Interactive Questions

Question 14) What is inheritance?

Inheritance allows one class to reuse another class.

Example

Employee → Manager

A Manager is an Employee.

The Manager inherits common attributes and methods from Employee.

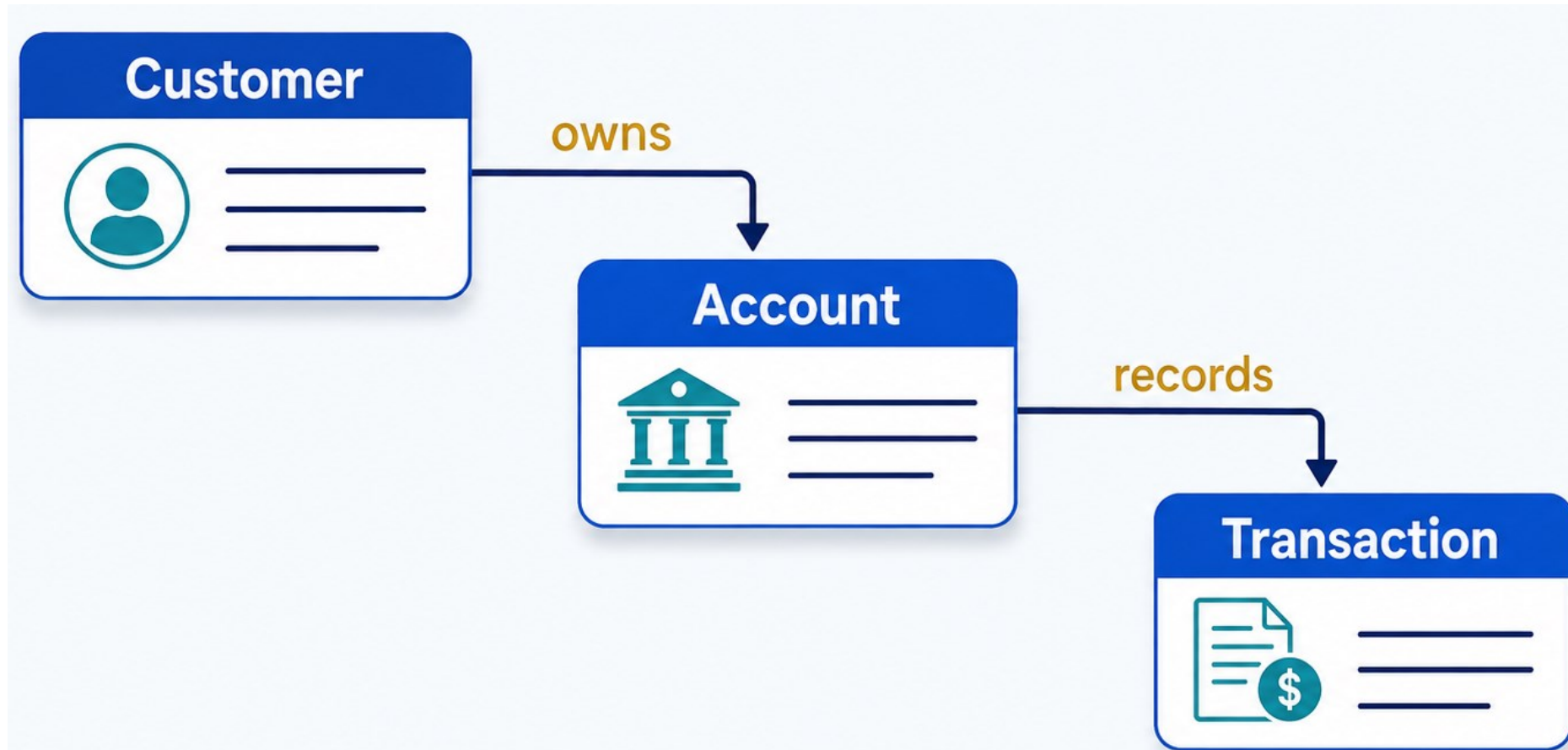
Lab 2: Interactive Questions

Question 15) What is a class relationship?

Classes communicate with one another.

Example

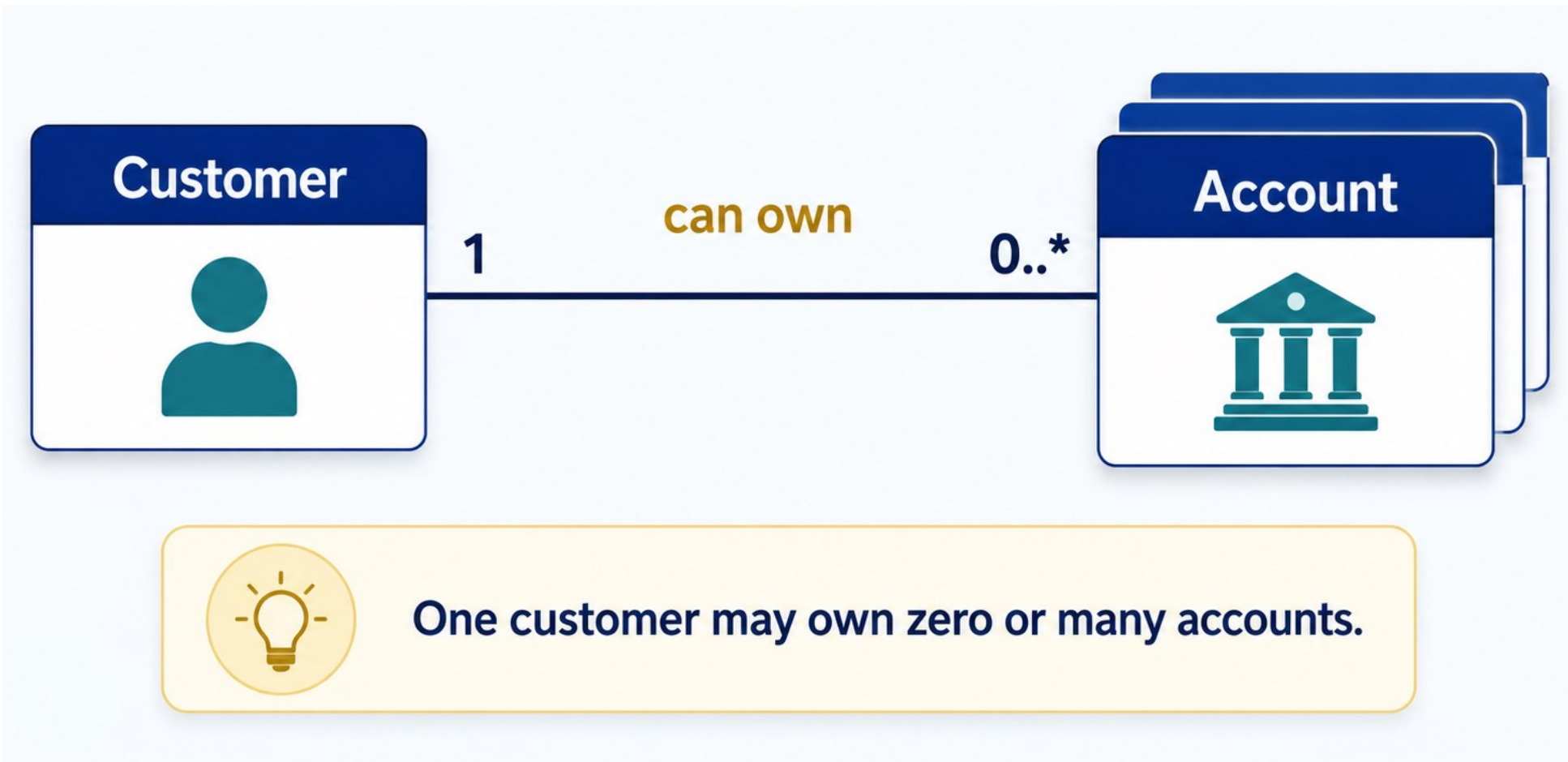
Relationships help describe how objects interact.



Question 16) What is multiplicity?

Multiplicity tells us how many objects participate in a relationship.

Example



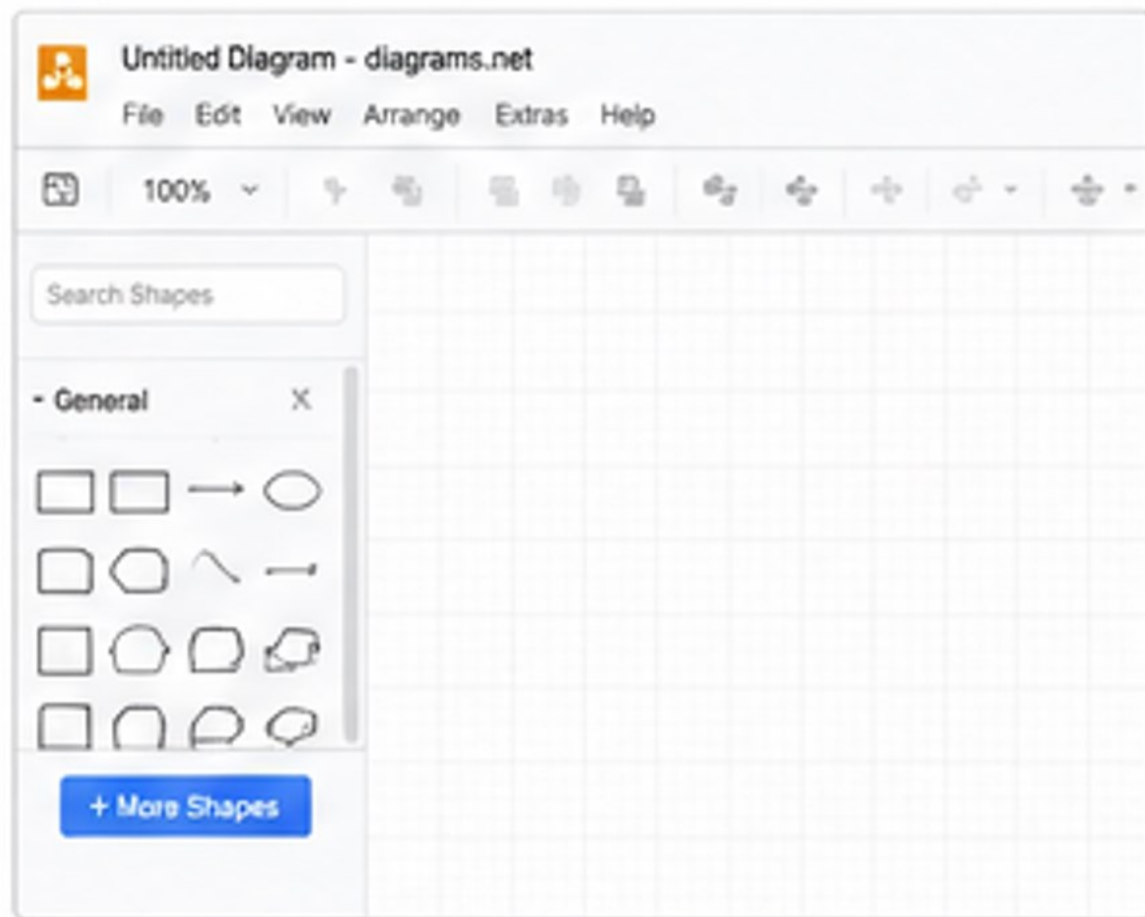
Lab 2: Interactive Questions

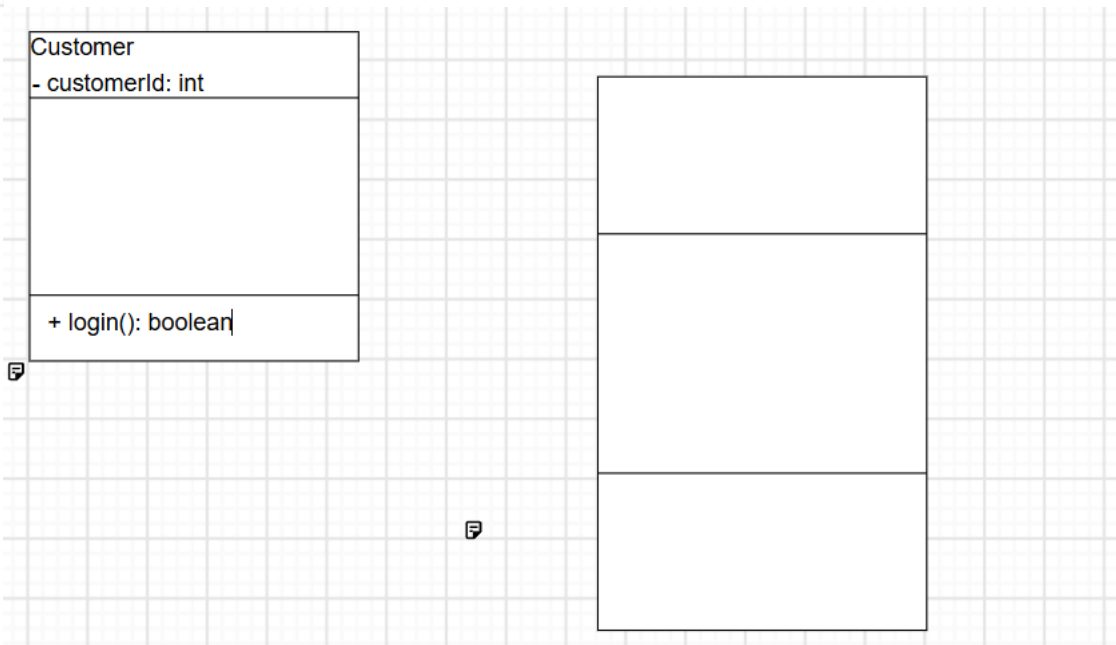
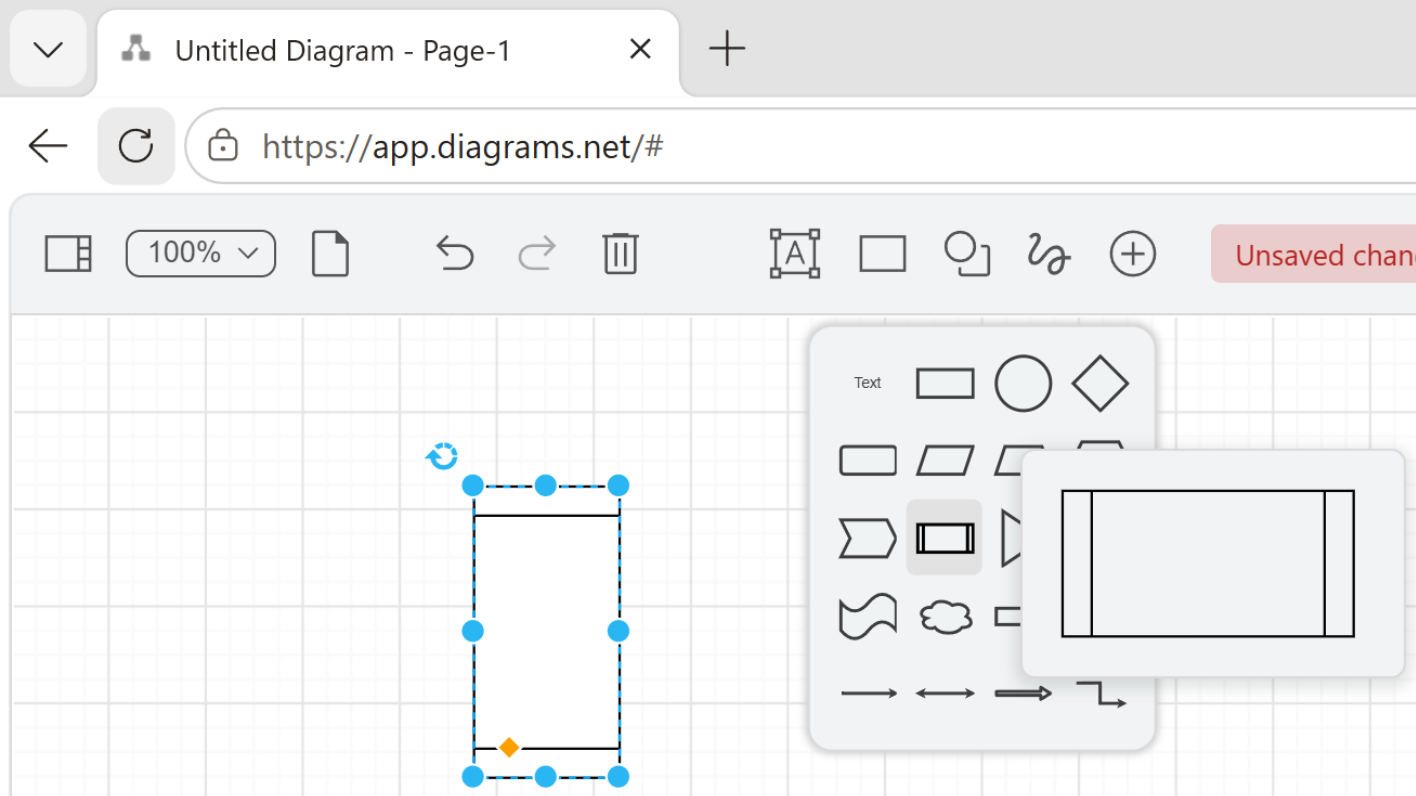
Question 17) How do we transform a Use Case Diagram into a Class Diagram?

Show the transformation steps:

1. Read the use cases.
2. Identify nouns → potential classes.
3. Identify actors → potential classes.
4. Identify behaviours → methods.
5. Identify data → attributes.
6. Identify relationships.
7. Add multiplicity.
8. Draw and review the UML class diagram.

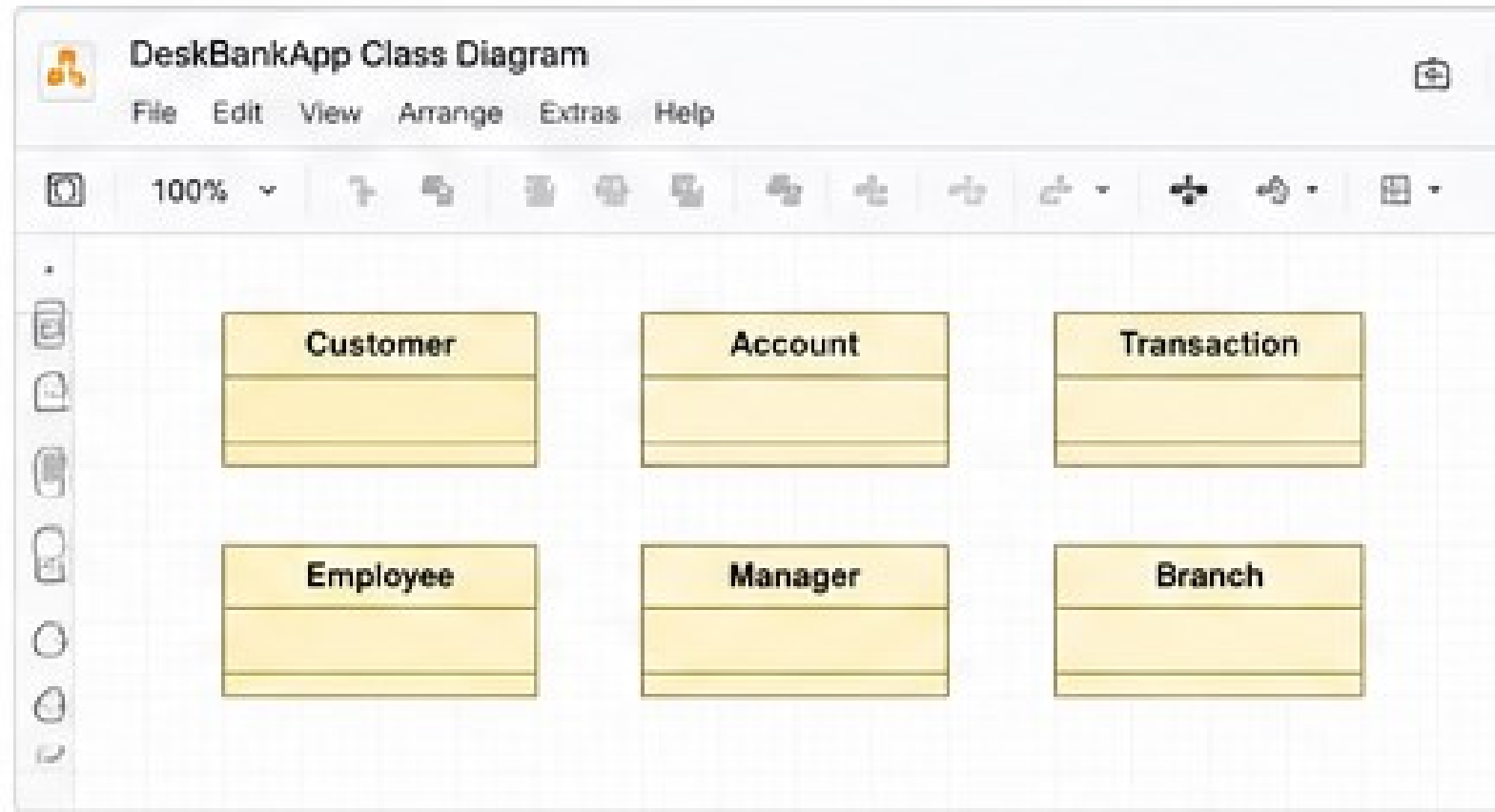
1 Open Draw.io





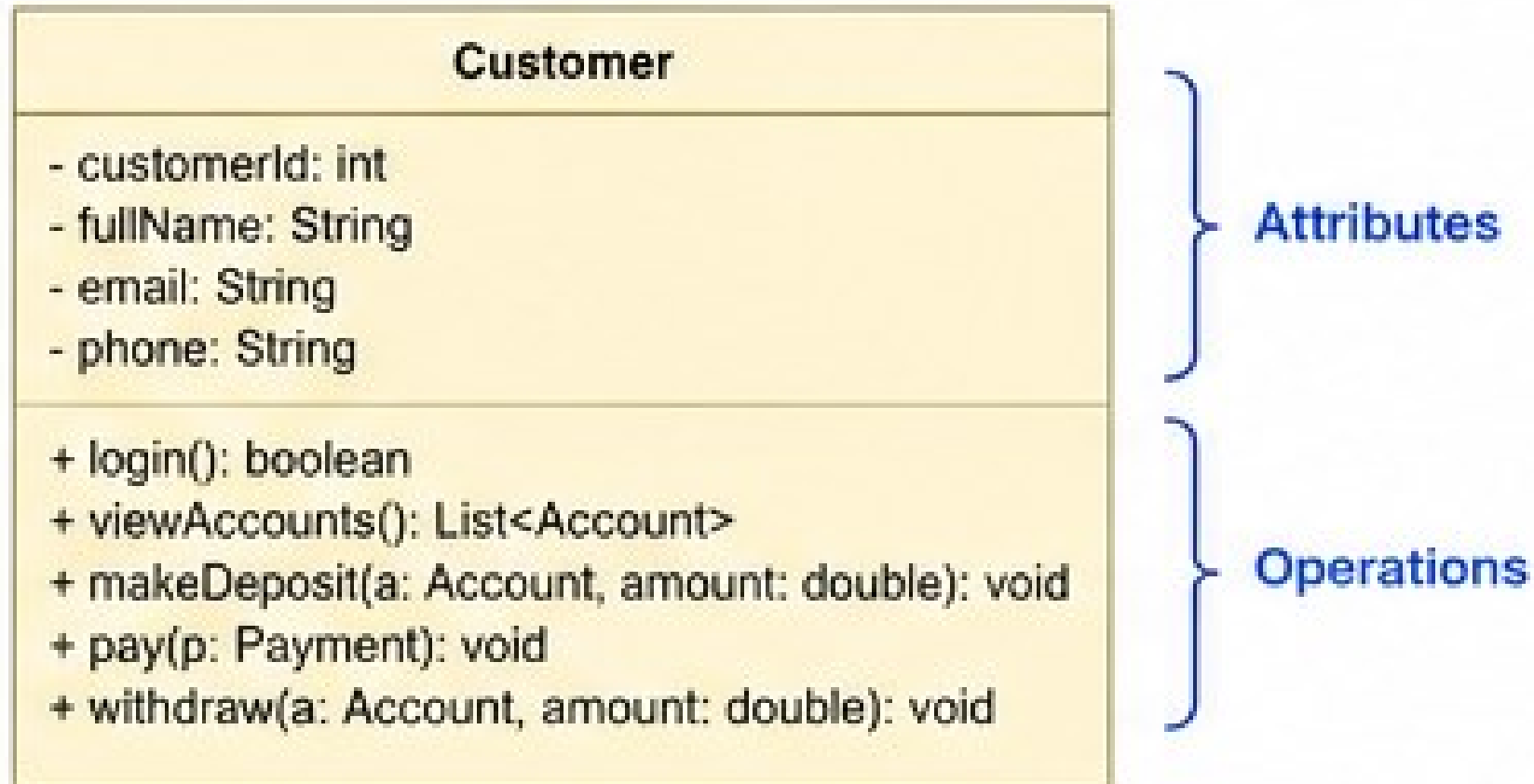
3 Add Classes

- Drag "Class" shape onto canvas
- Rename each class



4 Add Attributes & Operations

- Click inside each class
- Add **attributes** and **operations**
(Use **Ctrl+Enter** for new line)



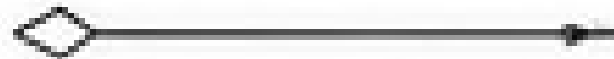
5 Add Relationships & Multiplicity

- Use **Connector** tool 
- Select a line style (Association, Aggregation, Generalization)
- Add multiplicity near each end

Association



Aggregation



Generalization (Inheritance)



1. Association

What is it?

An association means that two classes are connected or work together.

They know about each other, but one does not own the other.

Example) Customer ----- Account

Meaning

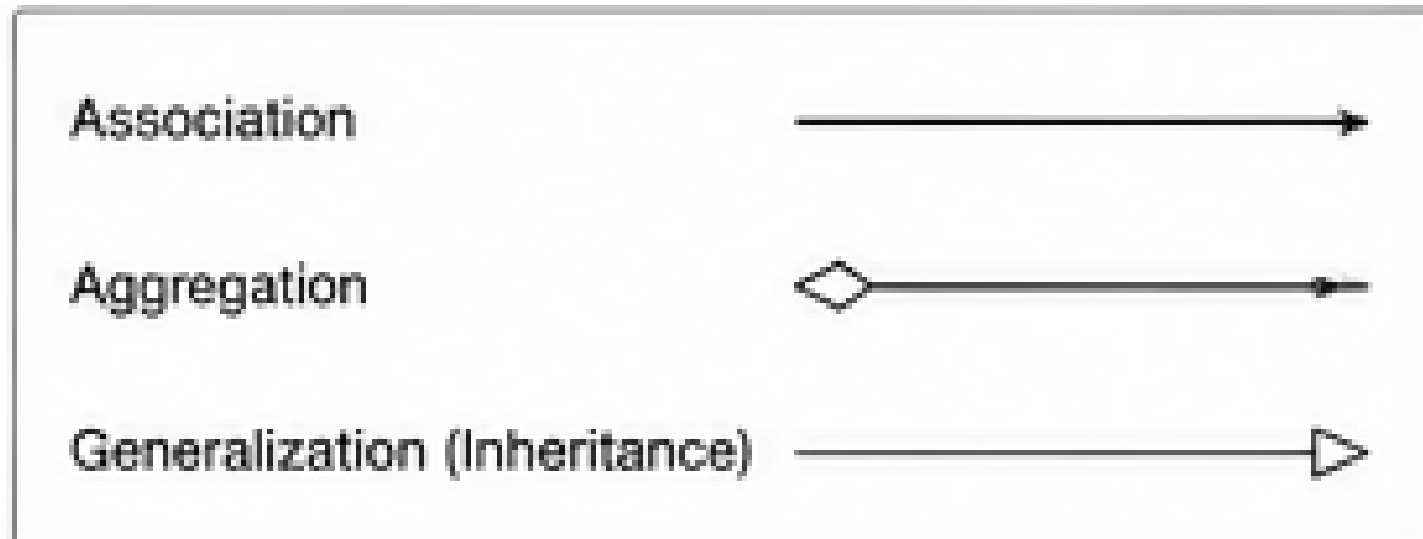
A customer owns or uses an account.

The customer and account are related.

Real-world example

- Teacher ↔ Student
- Doctor ↔ Patient
- Customer ↔ Account

Easy way to remember Association = "works with" or "uses".



2. Aggregation

What is it?

An aggregation means one object has another object, but the second object can still exist on its own.

This is a weak "has-a" relationship.

Example

Branch ◇———— Customer

Meaning

A branch has many customers.

If the branch closes, the customers still exist.

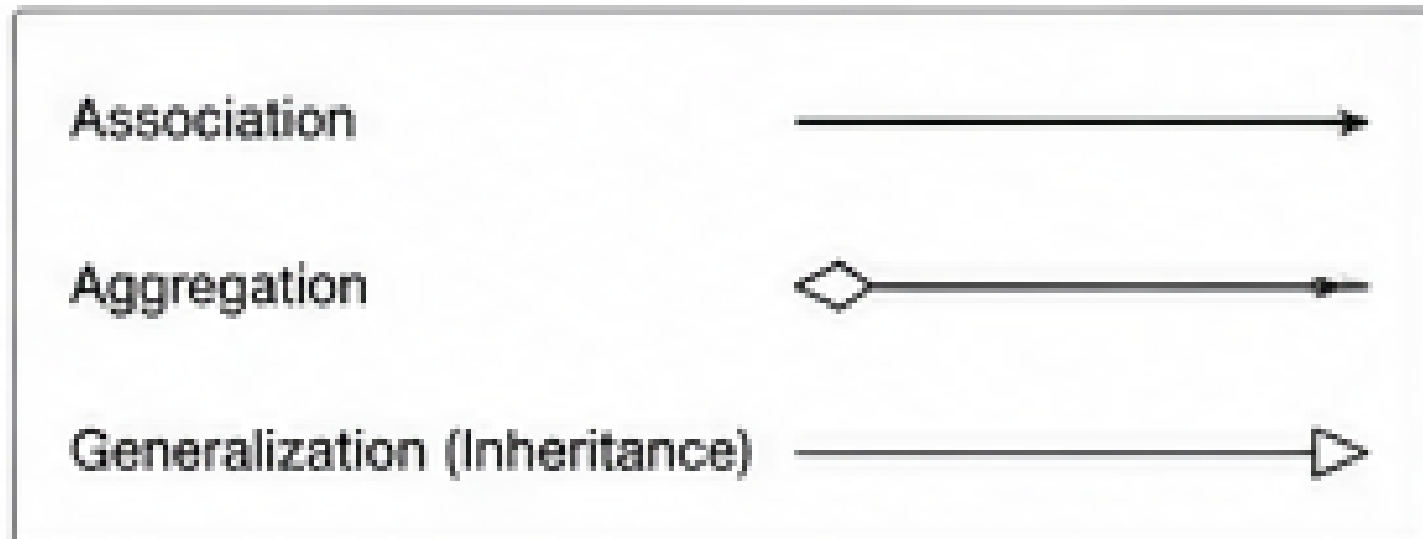
Another example

University ◇— Student

If the university closes,
the students still exist.

Easy way to remember

Aggregation = "has-a" (weak ownership). The part can live without the whole.



3. Generalization (Inheritance)

What is it?

Generalization means one class inherits from another class.

The child class automatically receives the parent's attributes and methods.

Example: Kangaroo -- Animal

Meaning

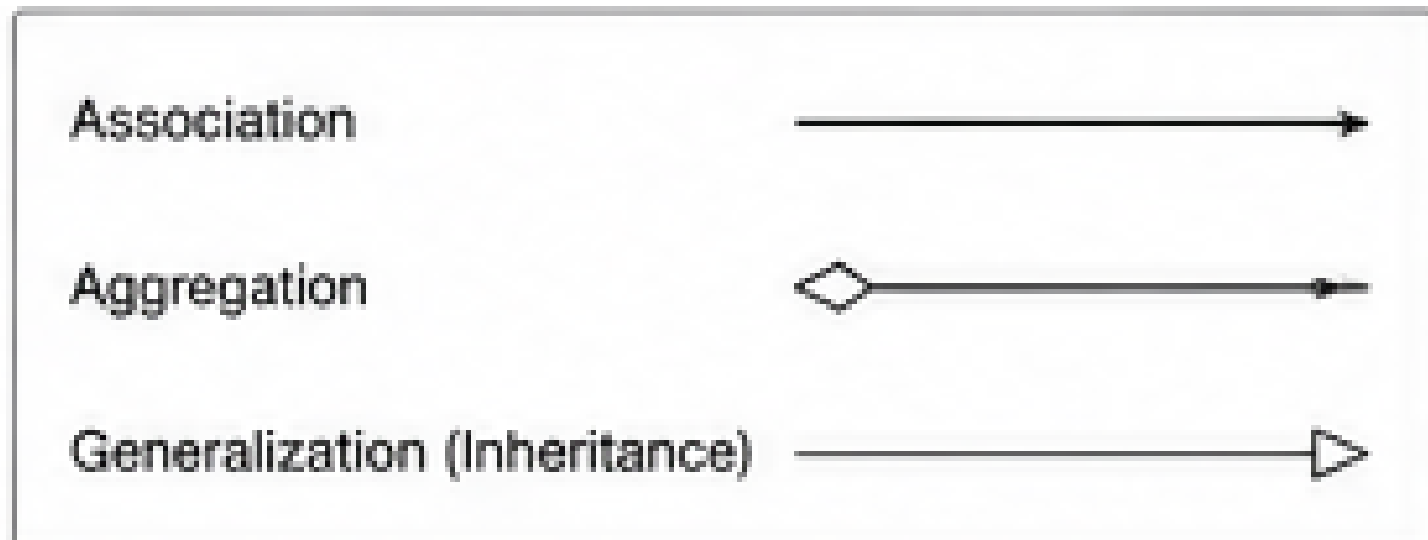
A Kangaroo is an Animal.

Kangaroo inherits:

- eat()
- breathe()
- sleep()

and adds its own method:

- jump()



Another example

Employee – Manager

Meaning

A Manager is an Employee.

The Manager inherits employee information and can also perform additional tasks.

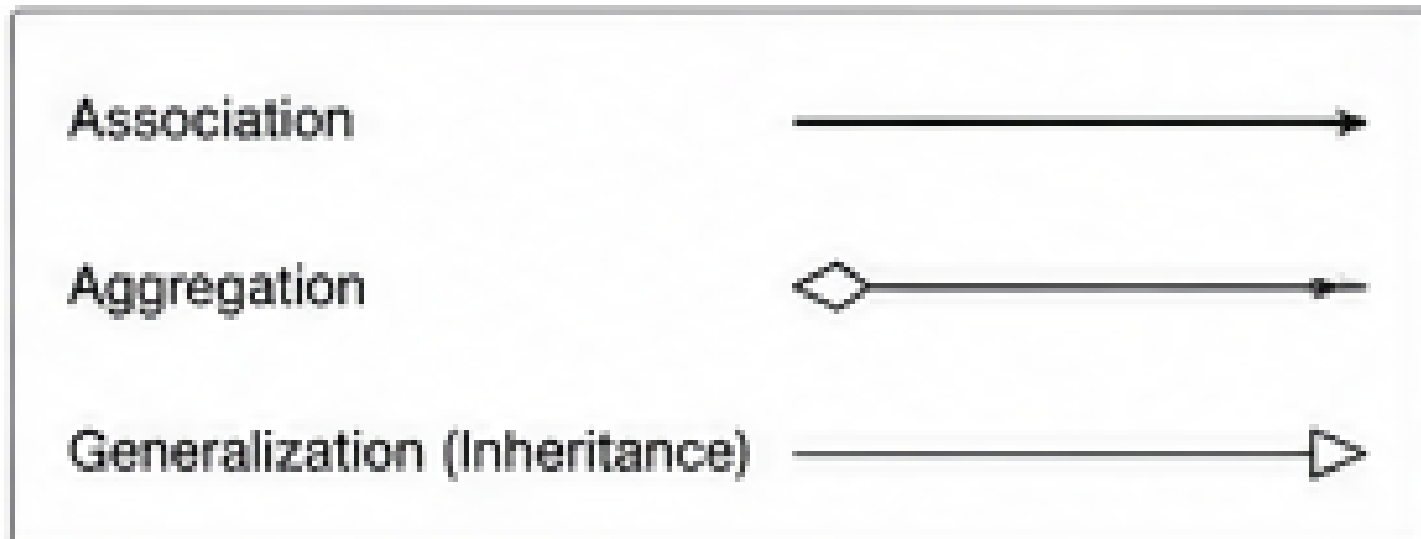
Easy way to remember

Inheritance = "is-a".

A Manager is an Employee.

A Dog is an Animal.

A Kangaroo is an Animal.



Quick Comparison

Relationship	Meaning	Example
Association	Works with	Customer ↔ Account
Aggregation	Has a (weak ownership)	Branch ◇— Customer
Generalization (Inheritance)	Is a	Manager △ Employee

Simple Memory Trick

Relationship	Remember this phrase
Association	Works with
Aggregation	Has a
Generalization	Is a

Q1. Which relationship should be used between Customer and Account?

Association

Because a customer uses or owns an account.

Q2. Which relationship should be used between Branch and Customer?

Aggregation

A branch has many customers, but customers still exist even if the branch closes.

Q3. Which relationship should be used between Manager and Employee?

Generalization (Inheritance)

A manager is a specialised employee.

Q4. Which relationship should be used between Kangaroo and Animal?

Generalization (Inheritance)

A kangaroo is an animal.

Key Takeaway

- Association → Classes work together.
- Aggregation → One class has another class, but the second class can exist independently.
- Generalization (Inheritance) → One class is a specialised version of another class and inherits its attributes and methods.

7 Save & Export

- File → Save As (diagrams.net)
- or File → Export As → PNG / PDF / SVG



Alternative tool: Visual Paradigm Community Edition

Best UML, SysML & ERD Tool [Forever FREE, Download]

www.visual-paradigm.com/download/community.jsp

70%

Visual Paradigm Desktop

VP Online

Contact Sales

Partners

Visual Paradigm

What's New

Features

Support

Pricing

Request Demo

Try Now

HOME

DOWNLOAD

Download Visual Paradigm Community Edition

FREE for non-commercial use only

Version: 18.1

Build number: 20260628

Download Visual Paradigm Community Edition

SSL Secure Connection

For Windows 64bit

Alternative tool: Visual Paradigm Community Edition

Best UML, SysML & ERD Tool [Forever FREE, Download] X

+

www.visual-paradigm.com/download/community.jsp?platform=windows&arch=64bit

70%

Visual Paradigm Desktop

VP Online

What's New

HOME / DOWNLOAD

Visual_Paradigm_CE_18_1_20260628_Win64.exe

13m left — 23.8 of 797 MB (962 KB/sec)

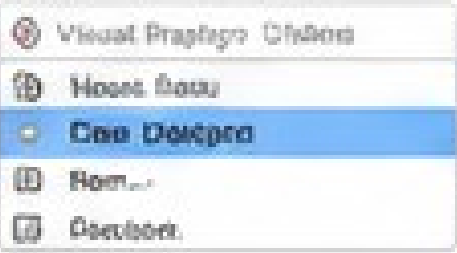
Show all downloads

Alternative tool: Visual Paradigm Community Edition

Option 2: Use Visual Paradigm Community Edition (Alternative)

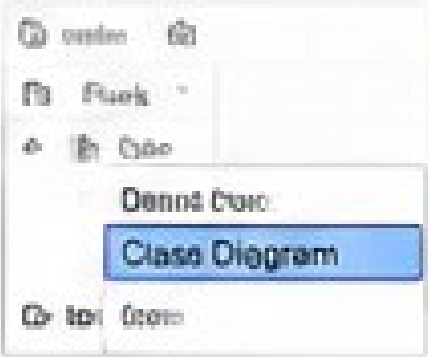
1 Create Project

File → New Project



2 Add Diagram

Diagram → New →
UML → Class Diagram



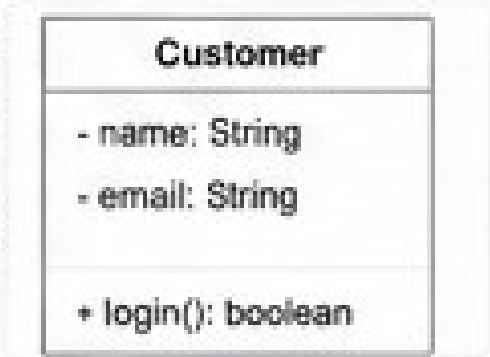
3 Add Classes

Drag Class shape
onto canvas



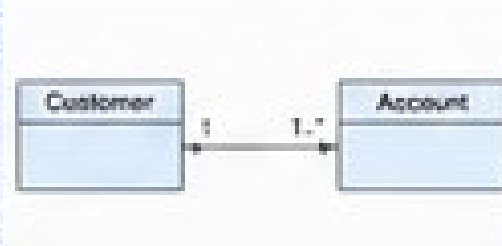
4 Add Details

Add attributes,
operations



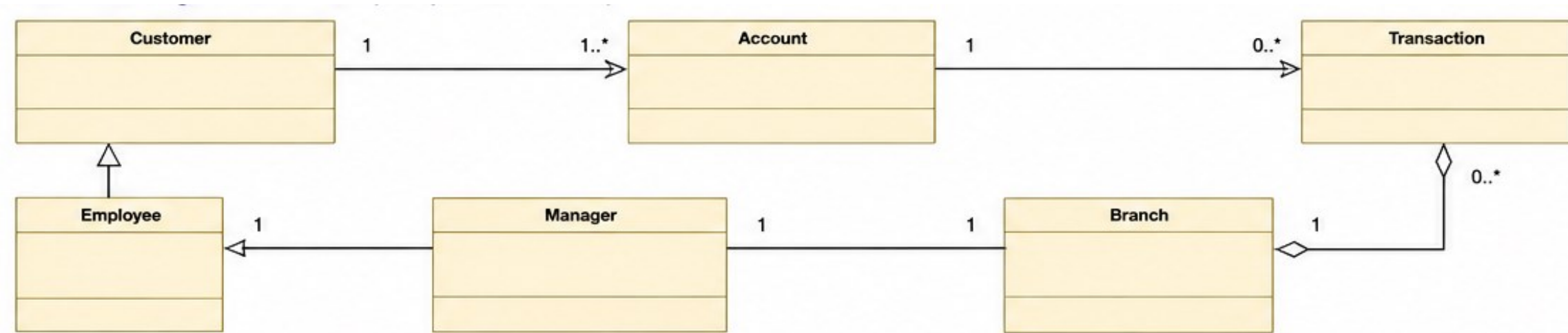
5 Connect & Save

Add relationships,
multiplicity, save



Partial UML Class Diagram – Group Design Challenge

Complete the classes and critically review the proposed relationships.



This diagram is intentionally incomplete, and some relationships may need to be changed or removed. Do not assume that everything shown is correct. Use the DeskBankApp requirements to justify your design decisions.

Your task is not only to fill in the class boxes. You must also decide whether each class, relationship and multiplicity is justified.

Your Group Task — 10–15 Minutes

Using the partial UML class diagram and the DeskBankApp requirements, work with your group to:

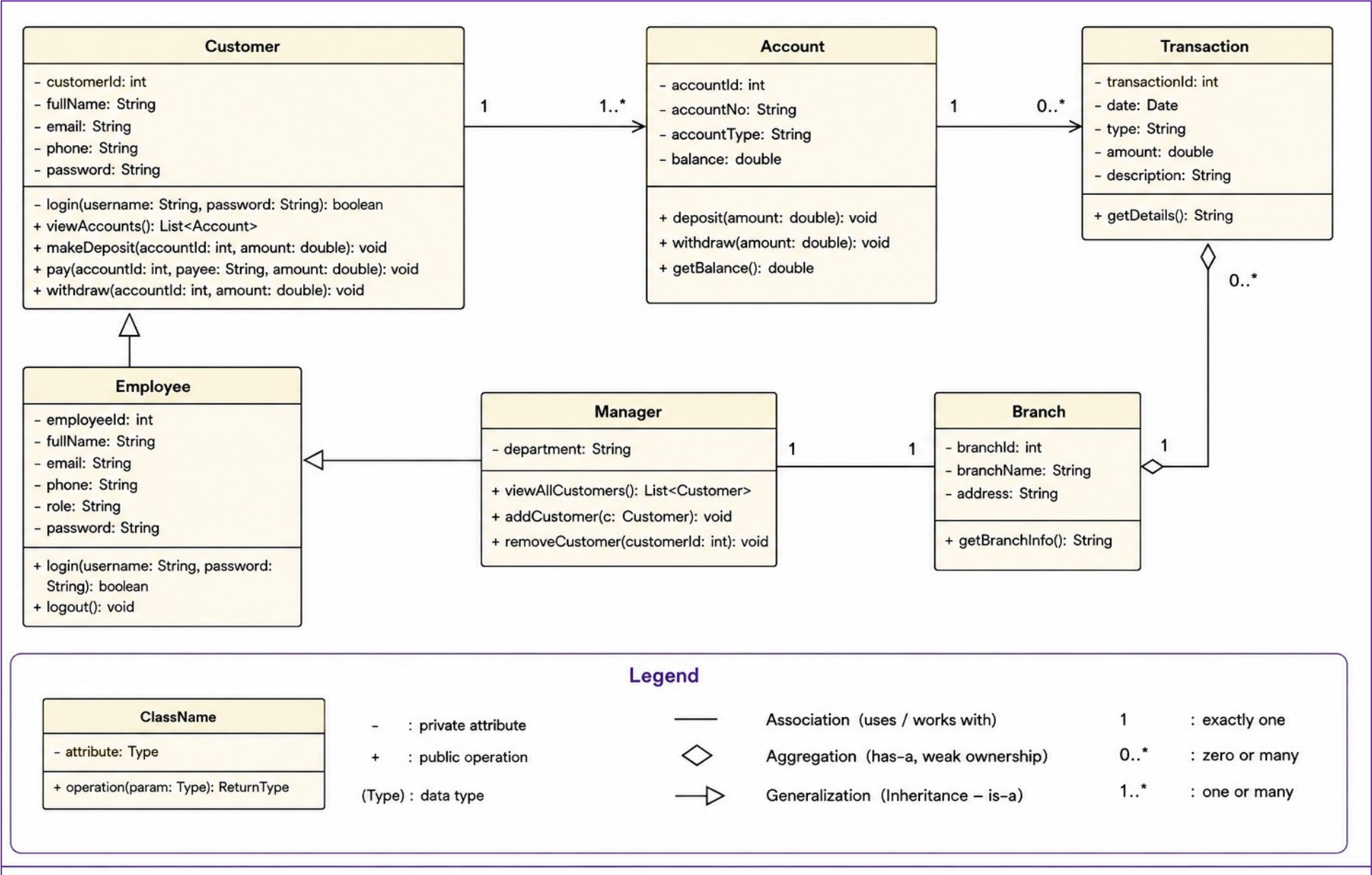
- Add suitable attributes and data types to each class.
- Add appropriate operations (methods), including parameters and return types where needed.
- Apply UML visibility symbols:
 - - private
 - + public
 - # protected

- Review the proposed relationships and decide whether any should be added, changed, or removed.
- Add suitable multiplicities, such as 1, 0..*, or 1..*.
- Check that every design decision is supported by the requirements.

Be ready to explain

- Why you selected each class.
- Why each attribute and method belongs in that class.
- Why each relationship and multiplicity is appropriate.
- There may be more than one acceptable design, but your decisions must be clearly justified.

Worked Example: UML Class Diagram for DeskBankApp



Customer Class

Question) What does the Customer class represent?

The Customer class stores information about each bank customer.

Attributes (Data)

- customerId
- fullName
- email
- phone

These attributes describe a customer.

Operations (Methods)

- login()
- viewAccounts()
- makeDeposit()
- pay()
- withdraw()

These methods describe what a customer can do.

Which of these are data?

- customerId
- fullName
- email
- phone

Which are actions?

- login()
- withdraw()
- deposit()

2. Account Class

Question) Why do we need an Account class?

A customer can own one or more bank accounts.

The Account class stores information about each account.

Attributes

- accountId
- accountNumber
- accountType
- balance

Methods

- deposit()
- withdraw()
- getBalance()

Where should the balance be stored?

Inside the Account class.

3. Transaction Class

Question) Why do we need a Transaction class?

Every banking activity should be recorded.

Examples

- Deposit
- Withdrawal
- Payment

The Transaction class stores these records.

Attributes

- transactionId
- date
- type
- amount
- description

Can one account have many transactions?

Yes.

4. Employee Class

Question) Why is Employee a separate class?

Employees use the system differently from customers.

Employees can

- log in
- log out

The Employee class stores employee information.

5. Manager Class

Question) What is special about the Manager class?

A manager is a specialised employee.

Managers can

- View all customers
- Add customers
- Remove customers

Managers have more permissions than ordinary employees.

Can every employee remove customers?

No.

Only the Manager.

6. Branch Class

Question) Why do we need a Branch class?

The system is installed in bank branches.

Each branch stores

- branchId
- branchName
- address

Understanding the Relationships

Relationship 1

Customer → Account

Question) What does this relationship mean?

A customer owns one or more accounts.

Multiplicity

1 ----- 1..*

Meaning

One customer has one or many accounts.

Can two customers share one account?

Depending on the system design,

Yes (joint account)

or

No

Always check the requirements.

Relationship 2

Account → Transaction

Question) Why is Account connected to Transaction?

Each account keeps a history of transactions.

Examples

Deposit → Transaction

Withdraw → Transaction

Payment → Transaction

Multiplicity

1 ----- 0..*

Meaning: One account may have many transactions.

Relationship 3

Employee $\leftarrow$ Manager

Question) What does the triangle mean?

It represents Inheritance.

Manager $\rightarrow$ is an $\rightarrow$ Employee

A manager inherits common employee information.

What information can Manager inherit?

- employeeId
- fullName
- email
- phone

Relationship 4) Manager → Branch

Question) Why is Manager connected to Branch?

A manager works at a branch.

One manager manages one branch.

One branch has one manager.

(Depending on the system design.)

Relationship 5

Branch ◇— Transaction

Question) What does the white diamond mean?

The white diamond represents aggregation.

Aggregation means a has-a relationship with weak ownership.

Example

A branch has many transactions.

If the branch is removed from the diagram, the transaction records may still exist independently.

Understanding Multiplicity

Question) What do these numbers mean?

1

Exactly one.

0..*

Zero or many.

1..*

One or many.

Example

Customer -> 1 -> Account -> 1..*

Meaning: One customer owns one or more accounts.

Review Questions) Q1

Which class stores the customer's balance?

Account

Q2) Which class stores deposits and withdrawals?

Transaction

Q3) Which class has methods such as deposit()?

Account

Q4) Which class can add customers?

Manager

Q5) Which relationship represents inheritance?

Manager → Employee

Q6) What does 1..* mean?

One or many.

Final Discussion) Is this the only correct UML Class Diagram?

No.

Software design is not always unique.

Different teams may produce different class diagrams.

As long as the design:

- satisfies the requirements,
- uses appropriate classes,
- defines sensible relationships,
- and can be justified,

it may still be a correct solution.

Lab 2: Interactive Questions

Question 18) Which use case creates the method deposit()?

Use case

Make Deposit → Method

deposit(amount)

The use case becomes behaviour inside the Account class.

The Make Deposit use case describes an action performed by the user.

This action becomes a method inside the Account class.

Java example

```
class Account {  
    private double balance;  
    public void deposit(double amount) {  
        balance = balance + amount;  
    }  
}
```

Example use

Java

```
account.deposit(100.00);
```

Python example

```
class Account:  
    def __init__(self):  
        self.balance = 0.0  
  
    def deposit(self, amount):  
        self.balance = self.balance + amount
```

Python

```
account.deposit(100.00)
```

Both examples add \$100.00 to the account balance.

Lab 2: Interactive Questions

Question 19) Why don't we copy the Use Case Diagram directly?

A Use Case Diagram answers: What should the system do?

A Class Diagram answers: How is the software organised?

They represent different views of the same system.

Lab 2: Interactive Questions

Question 20) Before submitting your UML Class Diagram, what should you check?

- ✓ Have you identified all important classes?
- ✓ Does every class have meaningful attributes?
- ✓ Does every class have useful methods?
- ✓ Are visibility symbols included?
- ✓ Are relationships shown?
- ✓ Are multiplicities included?
- ✓ Are inheritance relationships shown where appropriate?
- ✓ Is the diagram neat and easy to read?

Final Discussion Question) Imagine you are the software architect.

Can you think of any additional classes that would improve DeskBankApp?

Possible ideas:

- Loan
- CreditCard
- ATM
- LoginService
- Authentication
- Receipt
- Payment
- Notification

There may be multiple correct answers, provided students can justify why the class is needed and how it supports the system requirements.

Lab 2: Interactive Questions

Reflection Question) Wrap-Up

1. What is the difference between a class and an object?
2. Why do we use classes instead of one large program?
3. How do use cases help us identify classes?
4. What is the purpose of attributes and methods?
5. Why are relationships and multiplicity important in a UML class diagram?
6. If the bank later adds a Loan feature, which new classes might you introduce?

Let's Answer Your Questions



1. **Canvas Inbox** – For academic queries.
2. **Email** – For formal communication and attachments.
3. **Canvas Discussion** – For academic discussions and general queries.
4. **Reminder** – Please follow up if you do not receive a response within 3 business days.



Happy Learning and Bright Futures Ahead!

Dr. Farshid Keivanian

Stay Connected:

[linkedin.com/in/farshid-keivanian](https://www.linkedin.com/in/farshid-keivanian)

github.com/FarshidKeivanian

- *Education is the most powerful weapon you can use to change the world. — Nelson Mandela*